

FEASIBILITY STUDY

CampusPulse is a full-stack, AI-enhanced web application designed to automate and streamline campus placement drives. The system serves three primary user roles—Students, Recruiters, and Training & Placement Officers (TPOs)—through role-based dashboards. It introduces a Semantic Skill-Intelligence Engine powered by Sentence-BERT for contextual resume-JD matching, a Verification Pipeline for eligibility checking, and a Conflict-Detection Algorithm for preventing double-bookings across a multi-stage recruitment pipeline (Aptitude Test, Group Discussion, and Interview). Additional features include automated email reminders, CSV bulk import, PWA offline support, and an ECharts-based TPO dashboard with stage-wise funnel analytics.

This feasibility study evaluates the technical, operational, and economic viability of developing and deploying the Semester 1 Mini Project on a local server.

1. Technical Feasibility

Required Technologies

Frontend:

- Angular 17+: A robust, component-based framework for building responsive, role-based dashboards (Student, Recruiter, TPO). Angular provides built-in modules for HTTP clients, forms, and routing, making it ideal for enterprise applications.
- HTML5/CSS3: For structuring and styling web pages with a mobile-first approach.
- Tailwind CSS / Bootstrap: For rapid UI development with pre-built, customizable components.
- TypeScript: For type-safe, maintainable code.

Backend:

- Node.js: A lightweight, event-driven JavaScript runtime ideal for handling concurrent API requests.
- Express.js: A minimalist web framework for building RESTful APIs with robust routing and middleware support.

- JWT (JSON Web Tokens) : For stateless, secure authentication across user roles.
- bcrypt: For securely hashing passwords before storing them in the database.
- [Socket.IO](#): For real-time live system alerts (pop-up notifications) when recruiters schedule tests, GDs, or interviews.
- node-cron & Nodemailer: For scheduling automated email reminders (24-hour interview notifications, result declarations).

Artificial Intelligence & Machine Learning:

- Python (Flask) : A dedicated microservice for AI functionalities.
- Sentence-Transformers (all-MiniLM-L6-v2) : A lightweight Sentence-BERT model (80 MB) for generating contextual embeddings of resumes and job descriptions, enabling semantic matching.
- PyPDF2: For extracting text from uploaded resumes (PDF format).
- PyTorch: The underlying deep learning framework for running the Sentence-BERT model.

Database:

- MongoDB: A flexible NoSQL database for storing diverse data—student profiles, recruiter profiles, job drives, applications, and offer letters. Its aggregation pipeline is ideal for implementing the conflict-detection algorithm.
- MongoDB GridFS: For storing large files like resumes and offer letter PDFs.

Local Server:

- Node.js (v18+) & npm: For running the backend and frontend locally.
- Python 3.9+ & pip: For running the AI microservice.
- MongoDB Community Edition: For local database management.
- Git: For version control and collaboration.

System Architecture

- The CampusPulse system will use a monolithic architecture for the backend (Node.js + Express) with a separate Python microservice for the

AI engine (Sentence-BERT). The frontend (Angular) communicates with the backend via RESTful APIs.

- The architecture separates concerns as follows:
 - Frontend (Angular): Handles user interfaces, form validation, and API calls.
 - Backend (Node.js + Express): Contains business logic, routing, database interactions, and authentication.
 - AI Service (Python + Flask): Handles resume parsing, semantic embedding generation, and Explainable AI (XAI) feedback.
 - Database (MongoDB): Stores all application data including user profiles, job drives, applications, and chat history.
- For scalability and future growth, the system can be migrated from a local server to cloud-based hosting (e.g., Vercel for frontend, Render/Railway for backend, MongoDB Atlas for database) in the future.

Security Measures

- Password Encryption using bcrypt to store user passwords securely.
- Session Management using JWT (JSON Web Tokens) to prevent unauthorized access after login.
- NoSQL Injection prevention by using parameterized queries and Mongoose ODM with built-in sanitization.
- Role-Based Access Control (RBAC) to ensure students, recruiters, and TPOs have appropriate permissions.
- File Upload Validation to restrict resume uploads to PDF format and limit file size to 5MB.

Feasibility Conclusion (Technical)

This project is technically feasible using Angular, Node.js, Express, MongoDB, Python, and Sentence-BERT. The local server environment (Node.js, Python, MongoDB) is suitable for development and small-scale deployment. The lightweight Sentence-BERT model (80 MB) runs comfortably on a standard development machine. For large-scale or high-traffic scenarios, scaling to cloud hosting may be required in the future.

2. Operational Feasibility

User Experience

- Students will log in (accounts are pre-created by TPO via CSV; students activate via email link), upload their resumes, view eligible drives, apply with a single click, track their application status across the multi-stage pipeline (Aptitude → GD → Interview → Offer), receive Explainable AI (XAI) feedback on their shortlisting or rejection, view and accept/decline offer letters, and access the platform as a Progressive Web App (PWA) for offline schedule viewing.
- Recruiters will register (pending TPO approval), post job drives with detailed eligibility criteria, configure multi-stage pipelines (Aptitude Test and/or Group Discussion), view AI-ranked applications with XAI justifications, shortlist or reject applicants, schedule aptitude tests, GDs, and interviews (with conflict detection), declare results, upload offer letters (PDF), and track offer acceptance rates.
- TPOs (Administrators) will log in with full administrative privileges, approve or reject recruiter registrations after reviewing Trust Scores (WHOIS/MCA data), bulk-import student data via CSV, view advanced ECharts dashboards (placed vs. unplaced, stage-wise funnel visualization, offer-to-acceptance ratio, drop-off analytics), and oversee all active drives across all companies.

System Operations

- Database Management: TPOs can bulk-import student data via CSV. Admins can export placement reports (PDF/CSV) from the analytics dashboard. MongoDB provides flexible schema management for evolving requirements.
- Maintenance: Regular maintenance tasks include:
 - Database backups to prevent data loss (manual backups for local MongoDB).
 - Software updates to patch security vulnerabilities.
 - User support via FAQs and a support system for student and recruiter issues.

Deployment on Local Server

- The system will be deployed on a local server using Node.js, Python, and MongoDB for development and testing purposes.
- MongoDB Compass or the MongoDB shell will be used to manage the database for ease of administration.
- The Python AI service runs as a separate Flask application on a different port.

Feasibility Conclusion (Operational)

The system is operationally feasible on a local server. It allows for easy management of student data, placement processes, and results. The TPO dashboard provides actionable insights for decision-making. However, for broader adoption across multiple colleges, migrating to a cloud-based solution would improve performance, scalability, and accessibility.

3. Economic Feasibility

Costs Involved

Development Costs:

- Labor Costs: Student project (no external hiring or contractor fees required).
- Development Time: Estimated 8-10 weeks to complete the Mini Project.

Infrastructure Costs:

- Node.js, Python, and MongoDB Installation: Free and open-source.
- Database Hosting: If using a local server, costs are minimal (MongoDB Community Edition is free).
- Domain & SSL: Not required for localhost deployment. For future production deployment, costs must be considered.

Ongoing Costs:

- Regular maintenance, including security updates and database backups.
- Support for users (students and recruiters) via FAQs and helpdesk.

Revenue Streams (Future Commercial Potential)

- SaaS Subscription: Charge colleges or universities for using the platform (future multi-tenant deployment).
- Custom Solutions: Additional paid services, such as custom branding, detailed analytics, or user-specific reports.

Cost vs. Benefit

- The initial investment is low since the system is being developed using a local server and free tools (Node.js, Python, MongoDB, Angular, Git).
- As the platform scales and hosts more placement drives, the recurring costs will be manageable, particularly if hosting on cloud servers later.
- The system provides significant benefits: 80% reduction in manual effort for TPOs, better-qualified candidates for recruiters, and transparency for students.

Feasibility Conclusion (Economic)

The project is economically feasible. Initial development costs are virtually zero (all tools are free and open source). The platform has clear commercial potential if it acquires multiple college contracts for hosting placement drives in the future.

Overall Feasibility Conclusion

The CampusPulse Recruitment Automation System is feasible on a local server using Angular, Node.js, Express, MongoDB, Python, and Sentence-BERT. It is technically, operationally, and economically feasible with careful attention to security compliance and user adoption. The system can be developed within an 8–10 week timeframe and initially deployed on a local server for testing. Scalability to cloud hosting is possible once the system grows.

The project holds great potential to modernize and streamline campus recruitment processes, making them more efficient, transparent, and secure for students, recruiters, and placement officers.